

FasalDoc — Technical Architecture Document

1. System Overview

FasalDoc is architected as an **offline-first Progressive Web App (PWA)** backed by a **multi-provider LLM proxy**, with a **local-first data layer** that syncs to the cloud when connectivity allows. Every architectural decision traces back to one constraint: rural Pakistani users often have unreliable internet and low-end Android devices, so the system must degrade gracefully rather than fail outright.

See `FasalDoc_Architecture_Diagram.mermaid` for the visual flow referenced throughout this document.

2. Frontend

- **Framework:** React 19 + TypeScript + Tailwind CSS v4, bundled with Vite as a PWA.
- **Why a PWA over native:** one codebase installs on Android, iOS, and desktop directly from the browser — no app store approval cycle, critical for a fast-moving hackathon-to-pilot timeline, and zero install friction for farmers unfamiliar with app stores.
- **Offline behavior:** a service worker precaches core app assets on first load, so the app shell loads instantly even with no connection. Scan and chat data write to IndexedDB immediately; a sync queue pushes them to the backend once the device reconnects.

3. Backend

- **Runtime:** Node.js + Express.js, deployed as a dedicated service (`backend/src/server.js`) alongside the frontend on a single Vercel domain — one URL for the whole product, simplifying deployment and CORS configuration.
- **Core endpoints:**
 - `POST /api/diagnose` — accepts an image (and optional symptom checklist), returns disease name, matched database key, confidence score, and remedy.
 - `POST /api/chat` — accepts a conversation history and user message, returns an AI response scoped to agricultural topics.
- **LLM Proxy Layer:** the backend never lets the frontend call LLM providers directly (keeps API keys server-side only). It builds a structured prompt referencing known disease catalogue keys, calls the provider chain in order (Gemini → Groq → OpenRouter), and parses the response into a strict JSON schema before returning it to the client.
- **Fallback engine:** if every LLM provider fails (rate limits, outages, or connectivity issues on the backend’s own upstream calls), a local agricultural RAG (retrieval-augmented generation) engine answers from the curated remedy database directly — the user never sees a hard failure.

4. Data Layer

Store	Purpose	Sync Behavior
IndexedDB (Dexie.js)	On-device storage for scans, chat history, and recovery cases	Always written first; source of truth when offline
Supabase PostgreSQL	Authenticated, structured storage for users, history, and recovery tracking	Synced automatically when the device is online
Supabase Auth	User authentication	Includes a backend admin signup path for instant demo/testing login

Row-Level Security (RLS) is enforced at the database level in Supabase, so even if a client were compromised, a user’s queries are restricted to their own rows — this is a database-level guarantee, not just an application-level check.

5. AI Diagnosis Flow (step-by-step)

1. **Capture** — user photographs or uploads an image of the affected crop or animal.
2. **Submit** — the image (plus optional symptom checklist for livestock) is sent to `/api/diagnose`.
3. **Prompt construction** — the backend builds a detailed prompt that includes the known disease catalogue keys relevant to the crop/livestock type, so the LLM's output can be reliably matched back to a remedy database entry rather than free-text guessing.
4. **AI analysis** — the LLM returns a disease name, the matched catalogue key, a confidence score, and a brief description.
5. **Fallback** — if all providers are unavailable, the local RAG engine still returns a best-effort answer from the remedy database, so the feature never goes fully dark.

6. Why These Choices

- **Multi-provider LLM with automatic fallback**, rather than a single provider, protects against the single biggest operational risk for an AI-dependent product: any one provider's rate limits, outages, or pricing changes taking the whole diagnosis feature down.
- **Structured remedy database instead of free-form LLM-generated advice** is a deliberate safety choice — treatment guidance carries real financial risk for farmers if wrong, so the LLM's role is matching and explaining, not inventing dosages from scratch.
- **Local-first storage** isn't just an offline nicety — for the target user, it's the difference between a usable product and an unusable one, given inconsistent rural connectivity.
- **One Vercel domain for frontend and backend** trades some scalability ceiling for significantly simpler deployment, which matters more at pilot stage than at scale — this is a conscious "right-sized for now" decision, not a permanent architectural ceiling.

7. Known Scaling Considerations (for future phases)

- The model file/config baked into the backend service means updates currently require a redeploy; moving model artifacts to cloud storage (e.g. a Supabase bucket) would decouple model updates from code deploys.
- As user count grows well beyond pilot scale, the single-service Vercel deployment may need to split into separately scaled frontend/backend services.
- Rate-limiting and abuse protection (e.g. via a package like `express-rate-limit`) should be added before any public, non-pilot launch.